

paddle launch 方案设计

- 1、原则上请使用中文
- 2、文档放在agroup内
- 3、注意侧重阐述设计思路而不只是实现方案细节，体现对方案选型的利弊考量（必要的需要有预调研实验数据支撑），特别是对框架和用户两个维度的影响
- 4、注意多利用图表来阐述设计思路

paddle launch 设计文档

一、概要

##1、相关背景

- 分布式的启动需要用户关心 ip 等信息，易用性还有优化空间，很多 issue 来自 ip 配置错误
- 随着更多功能拓展需求的增多，分支变多且复杂变得混乱

##2、功能目标

- 用户易用性，让用户启动分布式根据便捷，屏蔽不需要用户关心的部分
- 支持多种功能的拓展，如容错、弹性、流程定制（auto-mapping）
- 和调度框架 operator 部分更深度、合理地协同

##3、方案要点

- 接口/易用性方面，提供屏蔽 ip 配置的启动方式
- 架构上借鉴 kubernetes 的优秀设计，对流程和模块进行抽象，增强拓展性
- 默认提供协同逻辑以应对更复杂的应用场景，如弹性等

##4、现有方案

目前支持的启动方式

参考 <http://agroup.baidu.com/paddlepaddle/md/article/3323588>

</>

Plain Text | 收起 ^

```
1 GPU training:
2   1. for single node training with all visible gpu cards:
3       fleetrn your_training_py (arg1 arg2 and all others)
4   2. for single node training with [0,4) cards
5       fleetrn --gpus="0,1,2,3" your_training_py (arg1 arg2 and all others)
6   3. for multiple node training such as two node:192.168.0.16, 192.168.0.17
7       on 192.168.0.16:
8           fleetrn --ips="192.168.0.16,192.168.0.17" \
9           your_training_py (arg1 arg2 and all others)
```

```

10         on 192.168.0.17:
11             fleetrun --ips="192.168.0.16,192.168.0.17" \
12                 your_training_py (arg1 arg2 and all others)
13
14 CPU training:
15     1. for single node training with multi servers and workers:
16         fleetrun --server_num=2 --worker_num=2 your_training_py (arg1 arg2
17         and all others)
18     2. for multiple node training such as two node:192.168.0.16, 192.168.0.17
19         \
20         with 2 servers and 4 workers.
21         on 192.168.0.16:
22             fleetrun --servers="192.168.0.16:6170,192.168.0.17:6170" \
23             --
24             workers="192.168.0.16,192.168.0.17,192.168.0.16,192.168.0.17" \
25             your_training_py (arg1 arg2 and all others)
26         on 192.168.0.17:
27             fleetrun --servers="192.168.0.16:6170,192.168.0.17:6171" \
28             --
29             workers="192.168.0.16,192.168.0.17,192.168.0.16,192.168.0.17" \
30             your_training_py (arg1 arg2 and all others)
31     3. use gloo backend for multiple node training such as two
32         node:192.168.0.16, 192.168.0.17 \
33         with 2 servers and 4 workers. (workers should set port)
34         on 192.168.0.16:
35             fleetrun --servers="192.168.0.16:6170,192.168.0.17:6170" \
36             --
37             workers="192.168.0.16:6171,192.168.0.17:6171,192.168.0.16:6172,192.168.0.17:6
38             172" \
39             your_training_py (arg1 arg2 and all others)
39         on 192.168.0.17:
40             fleetrun --servers="192.168.0.16:6170,192.168.0.17:6170" \
41             --
42             workers="192.168.0.16:6171,192.168.0.17:6171,192.168.0.16:6172,192.168.0.17:6
43             172" \
44             your_training_py (arg1 arg2 and all others)

```

- 支持模糊指定节点个数自动分配和预分配方式
- 基本设定是 collective 模式绑定 gpu，ps 模式绑定 cpu，但其实逻辑上无绑定关系
- 通过 workers 是否指定端口绑定是否使用 gloo，比较难懂
- 指定多机启动时 (--ips)，实际上的端口是固定的

启动调用逻辑

fleetrun 基本等价于 `python -m paddle.distributed.fleet.launch`

</>

Plain Text | 收起 ^

```
1 fleetrun
2 #!/usr/bin/python
3 # -*- coding: utf-8 -*-
4 import re
5 import sys
6 from paddle.distributed.fleet.launch import launch
7 if __name__ == '__main__':
8     sys.argv[0] = re.sub(r'(-script\.pyw|\.exe)?$', '', sys.argv[0])
9     sys.exit(launch())
```

入口

</>

Plain Text | 收起 ^

```
1 # 入口 launch.py
2 launch()
3
4 # 处理 argparse.ArgumentParser 模块
5 args = _parse_args()
6
7 # 判断启动模式 collective/ps
8 distribute_mode = which_distributed_mode(...)
9
10 # 如果启用 弹性, 使用 弹性 模块启动
11 if enable_elastic(args, distribute_mode):
12     launch_elastic(args, distribute_mode)
13 # 如果 collective 模式则使用 collective 模式启动, 否则使用 ps 方式启动
14 if distribute_mode == DistributeMode.COLLECTIVE:
15     launch_collective(args)
16 else:
17     launch_ps(args, distribute_mode)
```

collective

</>

Plain Text | 收起 ^

```
1 def launch_collective(args):
2     # 生成节点任务信息, 主要逻辑都在这里包括
3     # - 环境变量处理, 判断如 paddlecloud 等
4     # - 生成节点详细信息, 如多进程、端口分配等
```

```
5 # - 目前的模式 cpu/npu 逻辑也在这里
6 # - auto-mapping 逻辑生成环境变量
7 cluster, pod = get_cluster_info(args)
8 # copy 全局 env
9 global_envs = get_global_envs(args, tmp_dir)
10 # 启动工作进程
11 procs = start_local_trainers(...)
12 # 监控进程
13 while True:
14     alive = watch_local_trainers(procs, cluster.trainers_n ranks())
```

pserver

</>

Plain Text | 收起 ^

```
1 def launch_ps(args, distribute_mode):
2     # 封装
3     ps_launcher = ParameterServerLauncher(args, distribute_mode)
4     ps_launcher.start_ps()
5
6 class ParameterServerLauncher(object):
7     def __init__(self, args, distribute_mode):
8         self.args = args
9         ...
10    def start_ps(self):
11        cluster = Cluster(hdfs=None)
12        for node_rank, ip in enumerate(self.node_ips):
13            pod = Pod()
14            for i in range(len(self.server_endpoints_ips)):
15                server = Trainer()
16                pod.servers.append(server)
17                cluster.pods.append(pod)
18            self.start_pod_server(self.args, pod)
19            self.start_pod_worker(self.args, pod)
20            ...
21            self.procs["worker"][i].proc.wait()
```

二、意义

集中阐述本次升级的作用和意义。

要写明与团队及个人OKR的关系。

对于需要预调研数据支撑的需要依据预调研结论给出量化预估。

三、竞品对照

对照描述及点评主要竞品（包括但不限于tf、pytorch）与此相关的现状、趋势

四、设计思路与实施方案

##1、主体设计思路与折衷

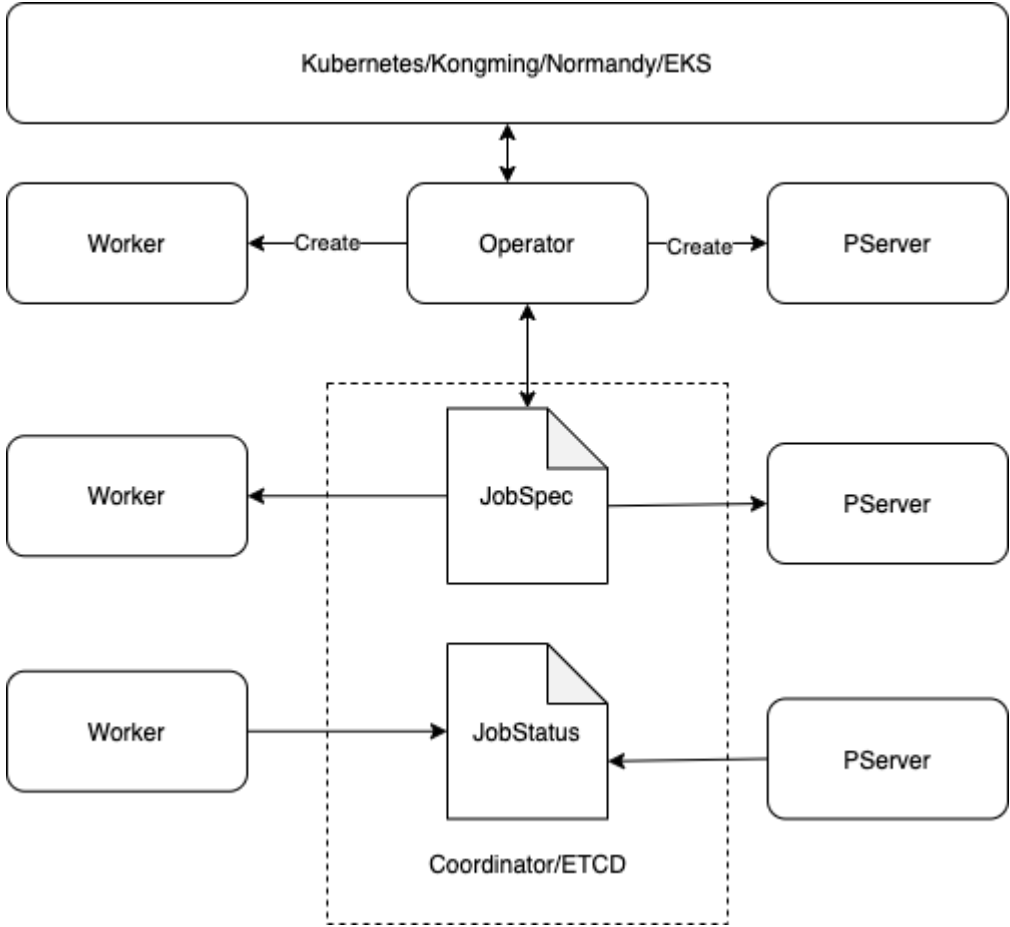
###整体全貌

launch 模块的定位

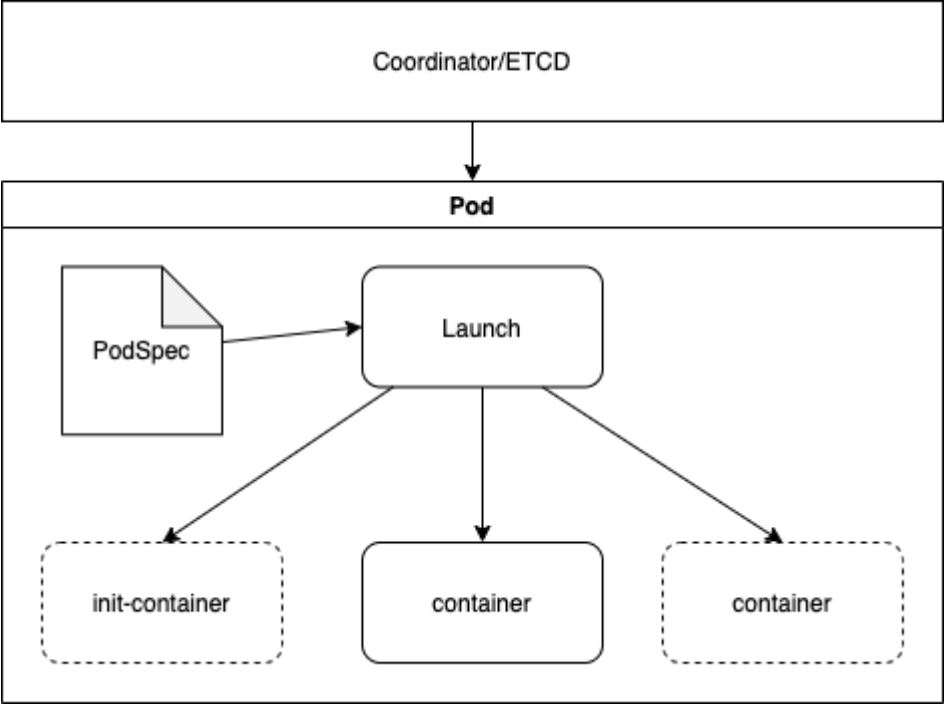
任务架构图

（这里主要展示的 coordinator 逻辑为 etcd 存储）

注：图中 worker/pserver 连线代表所有节点都有



工作节点（worker/ps）架构图



基本逻辑：

一个工作节点抽象为一个 pod，一个 pod 对应一个 launch，通过节点任务描述 PodSpec 执行对应操作

一个 container 对应一个进程

一个 pod 内可以并行运行多个 container（如 collective 多卡多进程）

init-container 在 container 运行前运行并退出（如 auto-mapping）

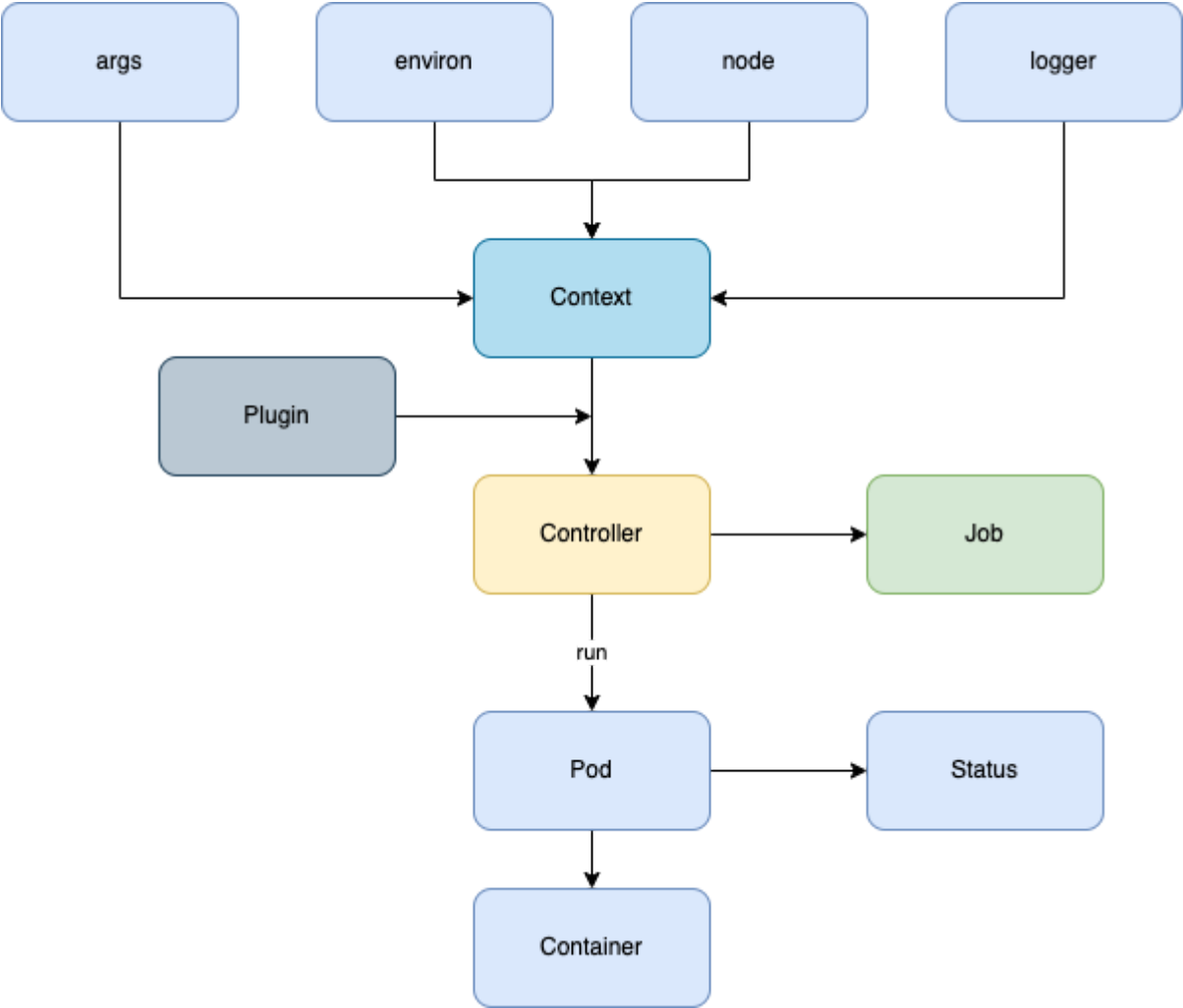
一个工作节点抽象为一个 pod，一个 pod 对应一个 launch，通过节点任务描述 PodSpec 执行对应操作

一个 container 对应一个进程

一个 pod 内可以并行运行多个 container（如 collective 多卡多进程）

init-container 在 container 运行前运行并退出（如 auto-mapping）

launch 实现架构



目录结构

</> Plain Text | 收起 ^

```
1 python/paddle/distributed/run/
2 |─ context
3 | |─ device.py
4 | |─ __init__.py
5 | |─ node.py
6 | |─ resource.py
7 |─ controllers
8 | |─ collective.py
9 | |─ controller.py
10 | |─ __init__.py
11 |─ __init__.py
12 |─ job
13 | |─ container.py
14 | |─ __init__.py
15 | |─ job.py
16 | |─ pod.py
17 |─ __main__.py
```

```
18 └─ plugins
19 |   └─ __init__.py
20 |     └─ ip.py
21 └─ utils
22     └─ kv_client.py
23     └─ kv_server.py
24     └─ process_context.py
```

因为整体重构，便于开发调试，新建目录，后续发布可以重命名为 launch。

重要概念

Context

Context 收集一切静态、初始化信息，在 launch 启动时就可以获得的有用信息。

在整个 launch 生命周期中为开发者获取信息提供 api。

主要包含以下组件：

- **args** 命令行参数解析
- **environ** 环境变量解析
- **node** 节点信息如，设备 gpu、npu 和 ip、port 等信息
- **logger** 统一日志服务，方便调试
- **plugin** 定义一个函数列表，用于静态调整信息，如环境变量兼容代码

此外，Context 接收统一的 **signal** 处理退出。

以上类和每个类的内容可以继续拓展。

Pod/Container/initContainer

Pod 即一个 launch 模块对应的节点，一个调度单元

Container/initContainer 是 Pod 运行核，是对**进程**的封装，一个 pod 可以包含多个

Container/initContainer，Pod 启动运行时，initContainer 先运行，成功退出后运行 Container；当有多个 Container 时，所有 Container 同步运行。

Controller

核心部分 Controller 是流程逻辑，通过面向对象设计

- **base controller** 作为基类提供基础函数封装和默认不需要改变的函数实现
- **collective controller** 提供 collective 启动方式下生成 job/pod 信息的逻辑
- **ps controller** 提供 ps 启动方式的控制逻辑

当有新流程需要实现时，理论上只需要开发新的 controller，实现特殊逻辑的函数即可。

比如以下两个函数必然需要重新定义：

</>

Plain Text | 收起 ^

```
1 # 判断是否使用该 controller 启动 pod
2 def enable(cls, ctx):
3
4 # 怎么定义 pod, 即执行逻辑
5 def build_pod(self):
```

一般的 controller 执行 pod 运行 run 时, 只需要吧 container 列表执行即可, 当需要 relaunch 时, 可能需要重写覆盖 run 函数.

Job/Status

Job 存储整个任务, 多个节点的信息, 可以已通过直接接收定义或者中心交换获得, 目前基本上只有 ip 信息, 后续可以根据需求拓展更多。

Status 存储目前 pod 甚至 job 运行状态信息。

后续如果考虑增加对外服务接口, 即封装访问到 Job 和 Status 对象即可。

启动方式

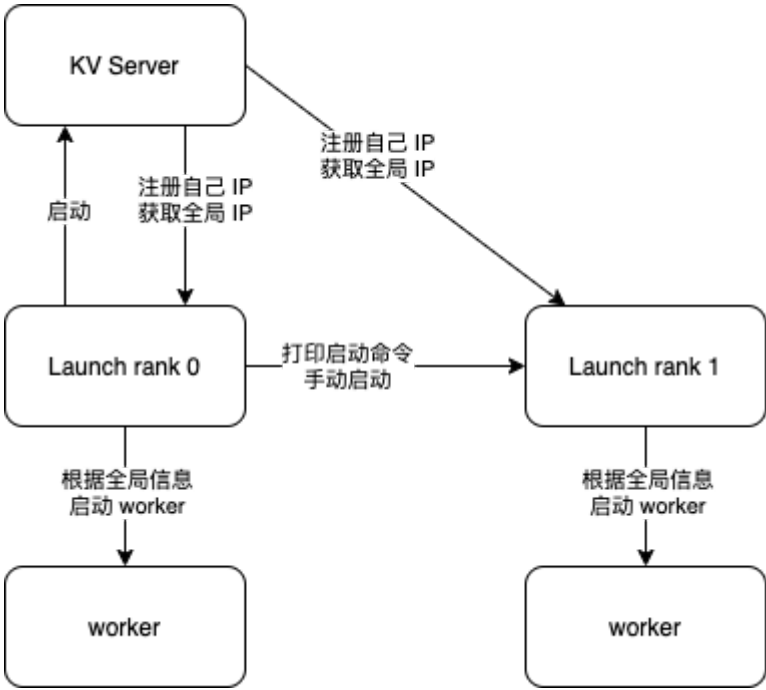
免 IP 启动方式

新增启动方式: 主节点上启动命令, 打印子节点启动命令, 然后复制命令粘贴到别的节点上执行即可。免去用户查看 ip, 配置ip

</>

Plain Text | 收起 ^

```
1 $ python -m paddle.distributed.run --np 2 demo.py
2 Copy the following command, and paste and run it in other node.
3 -----
4 python -m paddle.distributed.run --join 10.127.4.142:39774 --np 2 demo.py
5 -----
---
```



配置文件启动方式 (暂不支持)

</> Plain Text | 收起 ^

```
1 $ paddlerun -f demo.yaml
```

</> Plain Text | 收起 ^

```
1 # demo.yaml
2 PaddleJob:
3     name: demo
4     entrypoint: demo.py
5     nodes:
6         count: 3
7         ips:
8             - 10.10.10.1
9             - 10.10.10.2
10            - 10.10.10.3
```

- 如果只配主 ip，必须指定个数，所有节点通过此 ip 同步信息
- 支持单机分布式启动

单节点执行，命令分发（需求讨论）

单节点执行，配置所有节点信息？需要配合 ssh 免登陆，易用性存疑

方案对比总结

- container 对运行时的统一，不同流程采用同样的运行时封装

- context 对信息的统一处理，避免 if else 对不同类型判断
- logger 全局统一
- collective 支持单机模拟分布式
- ps 支持 ps-worker 成对部署

应用

####容错

容错功能的实现依赖以下部分

- 持久化存储：在进程失败本地或异地重启后能够获得继续参与训练所需信息，一般分为数据信息通过持久化的分布式存储挂载（即读写目录形式）和任务信息通过 coordinator 获取
- 程序的内部恢复机制：即程序重启后能够通过获得的信息恢复到之前的状态继续训练
- 信息协同：提供状态恢复所需信息，此处由 coordinator 提供服务
- 资源管理器：负责故障节点的资源释放和替换资源的分配，依赖 kubernetes 或其他

容错流程：以 worker 容错恢复为例

1. 故障发生时，launch 模块感知 进程异常，上报 coordinator；当节点故障导致 launch 退出时，由 coordinator 感知节点掉线
2. coordinator 做逻辑判断，条件满足（如超时）时，通知 launch 模块启动容错，当 launch 已退出时依赖调度资源模块分配资源拉起 launch 模块
3. launch 模块结束故障进程，根据最新信息重新拉起新的训练进程
4. 新进程启动，launch 感知进程健康上报 coordinator 恢复完成
5. coordinator 根据需要向节点同步最新的进度信息

故障期间，如果是同步模式，依赖该故障节点输出的其他节点将处于阻塞状态；如果是异步模式，其他节点的流程不受影响

弹性扩缩容

结合上面的模块定义，可实现如果扩缩容流程

外部扩缩容流程：

1. 当外部有扩容或缩容需求时，通过 operator 发起修改节点变化信息的请求，pod 将会被新建或删除
2. coordinator 感知节点变化（etcd 中 jobStatus 信息被更新），更改任务状态；
3. 各个节点上的 launch，通过 watch 机制实时感知当前最新信息，更新节点内 podSpec 变化
4. 根据 podSpec 变化执行预定义操作

内部扩缩容流程：

1. 当内部感知需要扩容或缩容时，向 coordinator 发起请求（修改 etcd 中 jobSpec 信息），更改任务状态；
2. coordinator 向 operator 发起增删节点请求
3. 后续同上

##2、关键技术点/子模块设计与实现方案

逐点列出。对每个技术点/子模块要有清晰描述，不要只附参考文献。涉及数据结构、接口设计的要明确列出对应方案，修改或新增代码的位置。每一点若有多种实现可能的，请特别列出选型考量因素（必须时需要预调研实验支撑）

##3、主要影响的模块接口变化

###请列出核心设计对应的直接接口变化

###请逐一排查对对框架各环节的影响

包括不限于网络定义、底层数据结构、OP、数据IO、执行、分布式、模型保存、预测部署各环节的影响。如没有影响也要注明。

##4、测试和验收的考量

自测方案，CE，目标达成验收的度量方式。

五、示例

简单例子

</> Plain Text | 收起 ^

```
1 python -m paddle.distributed.run demo.py
2
3
```

六、影响和风险总结

对用户的影响

对框架架构的影响

对性能的影响

##对与竞品差距与优势的影响

其他风险

七、规划排期

时间和人力规划，主要milestone

八、待讨论

需要进一步讨论的问题，开放性问题，有争议问题

九、评审意见

1	问题	提出人	处理说明	状态
2				
3				
4				

名词解释

#附件及参考资料

Collective模式Fleet分布式API设计方案

<http://agroup.baidu.com/paddlepaddle/md/article/1650990>